UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

COS 301 Software Engineering

Capstone Project: Demo 1 Instruction

A Singh, O Welsh & K Homan

Copyright © UP 2025– All rights reserved.

1 Introduction

The First Capstone Demo is scheduled for 22nd May 2026. Please ensure you book **a demo slot on Hyperperform**. For Demo 1, you must have **at least three use cases** of your capstone project implemented. While we want to see your system's wireframe and requirements, we expect to see **light implementation**. Show the working components of your system during the Demo. You may and should make use of mocks to demonstrate that the use case works in the event that it uses data or logic from components that have not yet been implemented.

As discussed during the capstone presentation in one of our lectures, all teams must schedule a 5 to 15 minutes meeting at least twice a week as part of your stand-up meetings (internal team meetings). After each meeting, team members should log meeting minutes in the Group Discussion for each demo on ClickUP. It aims to provide a brief update on your project progress, identify obstacles, and plan your tasks. Missing this meeting may result in project delays. Be sure to include for each meeting the date, time and attendees of each meeting. Please type this out on the ClickUP Page, do not upload a PDF or document for the following format:

```
Date: 21 June 2026 @ 13:00
Team Members:
- List of attendees
Duration: 10 minutes
Discussion Points:
 - What the meeting talked about
 - High-level bullet points are fine
Action Items:
 - What parts of the system are going to be worked on
```

COS 301 lecturers will evaluate these minutes and use them to scale team and individual marks for each demo. Team leaders must ensure team adherence to this schedule.

You have multiple clients for your Capstone project. Schedule a meeting with your industry client as soon as possible. Regular meetings (weekly or bi-weekly) with your client are expected to be held using a medium of your choice (e.g., Google Meet or Discord). You must meet the client's needs as your project progresses and respond to any feedback.

Your industry mentor grades you throughout the Demos. The team leader must forward the grading link to your mentor. **It is the team leader's responsibility to ensure your industry mentor grades your team on or after the Demo day (at most 3 days after Demo day).**
Please note that attendance by your mentor is not compulsory. Here is the link for the mentor to grade your team:
`https://forms.gle/CVsSfghrb77W69gE9`

No marks will be added from the industry mentor once the marks are published.

For industry mentors who choose to attend the demos physically, use the following link (this must be completed 3 working days before the demo): `https://forms.gle/Z7WvAiGBAu8RaFSaA`

If an unsolvable dispute, issue, or challenge arises with your team member, client, or industry mentor, you **must** email the staff immediately. Delaying these disputes may impact your project's final output. COS301 does not allow any student to have a "free ride." Note: only doing documentation for a demo is considered a "free ride," and you will fail.

Once you receive your assigned capstone project on Hyperperform, here are the next steps:

- Set your first requirement clarification meeting with your client as soon as possible. Your client's details are on Hyperperform as well as the project proposal.
- In the email to your client, ensure you cc all COS 301 staff. **Please DO NOT cc Ms Willemse in your email.**
- Once the meeting is scheduled, send a Google Meet invite to all lecturers (if the meeting is not on Google Meet) and include the meeting link in the calendar invite.
- All first requirement meetings must be attended by one or more COS 301 staff members. The meeting should happen between the 20th and 30th of April 2026.
- Teams failing to meet the requirement clarification session within the specified time will be penalised. Team leaders will bear the highest penalty if the meeting is not held before Demo 1.
- A reminder that at least two (2) stand-up meetings should take place every week with your team.

2 In-person Demo

1. For Demo 1, you are expected to:
    - In-person Demo on campus at IT 4-66. A link to book slots will be sent out later.

- Your GitHub landing page (README) should include:
  - A short description of your project: "TeamName - ProjectName - Project Description."
  - A link to the Functional Requirements (SRS) document.
  - A link to your **GitHub Project Board**.
  - A short profile of each team member, including a LinkedIn link visible in your GitHub.

2. Your GitHub repository should show the following:

   - Git structure (mono repo), Git organization, and management.
   - Branching strategy.
   - Code quality badges.
     (a) Code Coverage (Coveralls, Codecov, SonarCloud).
     (b) Build (AppVeyor, GitHub Actions, CircleCI, Drone).
     (c) Requirements (shields.io).
     (d) Issue tracking (GitHub Issues).
     (e) Monitoring (NodePing, Uptime Robot).

3 Configuration Settings

IMPORTANT!!! This subsection defines the required configuration and tooling standards for version control within the project. Adhering to these settings ensures traceability, accountability, and consistency across all team members.

- **Git User Configuration:** All team members must configure their local Git environment such that their `user.name` matches their GitHub username. This ensures that all commits can be accurately attributed and audited.

```
git config --global user.name "your-github-username"
git config --global user.email "github@email.com"
```

- **System Git Requirement:** All version control operations must be performed using the system-installed Git via the command line interface. Graphical Git clients (e.g., GitHub Desktop, GitKraken) are not permitted, as they often rely on bundled or isolated Git instances rather than the system Git. This can interfere with commit tracking and auditing mechanisms used in the project.

  Failure to comply with this requirement may result in commits not being properly tracked. This loss of tracking is not recoverable or subject to appeal, and untracked contributions will not be considered for assessment.

- **AI Usage Tracking:** Team members are required to install the COS301 VibeCheck tool. This tool is used to monitor and track the use of AI-assisted code generation within the repository, ensuring transparency and accountability in the development process. This is a hard requirement. Any commits made without having this tool active will **not** count towards your contribution, once the installation instructions have been released.

  NOTE: More information will follow soon regarding the setup and usage.

  Vibe coding (i.e., indiscriminately generating large portions of code using AI without clear understanding, validation, or meaningful contribution) is strictly prohibited. Students are expected to demonstrate ownership of their work, including the ability to explain, justify, and maintain any code submitted.

  Marks are awarded based on demonstrated understanding and individual contribution. Where work is determined to be predominantly AI-generated without sufficient student input or comprehension, will result in reduced or zero marks for the affected contributions.

4 Deliverable

Your project's deliverables and tool usage will be evaluated for every Demo. All artefacts should reflect the current project state. The master/main branch must always reflect a working version of your system, and you must merge into the master/main before each Demo. Documentation must be a PDF or markdown on the repository, providing Google Drive links, or Overleaf will not be allowed.

4.1 Requirements Specifications

The primary deliverable for Demo 1 is the base SRS for your project. Follow the mini-project SRS as a guideline. **You do not need a complete or detailed SRS.** An incremental approach, as applied to Agile methods, is advised. Update the SRS document over time, placing old sections that have changed in an appendix. The document should contain:

- Introduction
- User Stories / User Characteristics
- Functional Requirements
- API Service Contracts
- Domain Model
- Architectural Requirements:

  - Quality Requirements
  - Architectural Patterns
  - Design Patterns
  - Constraints
- Technology Requirements

4.1.1 Introduction

State the business need for the application and summarize the project's scope.

4.1.2 Domain Model

Provide a valid UML class diagram illustrating the software solution.

4.1.3 User Stories / User Characteristics

List all intended users and explain the system's usage for each.

4.1.4 Use Cases

Draw high-level use case diagrams for your system's use cases.

4.1.5 Functional Requirements

Specify the functional requirements to satisfy the use cases. Assign the requirements to subsystems.

4.1.6 Quality Requirements

Specify and quantify the system's quality requirements, such as performance, reliability, scalability, security, and maintainability.

4.2 Design Specifications

This section defines the visual, structural, and interaction design decisions that guide the development of the system. It serves as a blueprint to ensure consistency across different components and teams, particularly in a modular development environment. The design specifications translate system requirements into concrete interface and user experience guidelines, reducing ambiguity during implementation. This section typically includes branding guidelines and interface representations such as wireframes.

4.2.1 Brand Style

The brand style defines the visual identity of the system and ensures a consistent and professional appearance across all interfaces. It provides guidelines that all developers and designers must follow when implementing the user interface.

Key elements include:

- **Color Palette:** A defined set of primary, secondary, and accent colours, including their HEX or RGB values. These colours should be selected with accessibility in mind, ensuring sufficient contrast for readability.
- **Typography:** Standardised font families, sizes, and weights for headings, body text, and other UI elements. This ensures readability and a clear visual hierarchy.
- **Logo and Iconography:** Guidelines for the use of logos, icons, and graphical elements, including sizing, spacing, and placement rules.
- **Design Principles:** High-level principles such as consistency, simplicity, responsiveness, and accessibility that guide the overall look and feel of the system.
- **UI Component Styling:** Standard styles for common interface elements such as buttons, forms, navigation bars, and modals to ensure uniform implementation across the system.
- **Accessibility:** Guidelines to ensure the interface is usable by individuals with diverse abilities, including sufficient colour contrast, keyboard navigability, screen reader compatibility, and adherence to accessibility standards (e.g., WCAG).

4.2.2 Wireframes

Wireframes provide a visual representation of the system's user interface structure. They focus on layout and functionality rather than detailed visual design, serving as a guide for both developers and stakeholders.

Key elements include:

- **Screen Layouts:** Low- to mid-fidelity representations of key system screens such as login pages, dashboards, and feature-specific views.
- **Navigation Flow:** Illustrations of how users move between different screens, often represented using arrows or flow diagrams.
- **Component Placement:** The positioning of interface elements such as menus, buttons, forms, and data displays on each screen.

- **User Interaction Points:** Indications of where users input data, trigger actions, or receive system feedback.
- **Annotations:** Brief notes explaining the purpose and behaviour of specific elements or interactions to clarify design intent.

5 Base Features

A base set of features is required from all teams for Demo 1, and these do not count towards your total use cases:

- Registration and Login
- Basic Themes
- Form Validation

6 Assessment Rubric

DEMO 1 RUBRIC	
Component	**Mark**
Demo Mark	
Project Overview (1 min)	2
Project Plan (2 min)	3
Live Demo – Features (5 min)	20
Live Demo – Continuous Integration (CI) + Repo Setup + Testing (2 min)	10
Q&A (5 min)	10
Overall Demo Presentation	10
Documentation Mark	
SRS - Functional Requirements	10
SRS - Use Cases + Use Case Diagrams	10
SRS - Domain Model	10
SRS - Preliminary Architecture	5
Design - Brand Style Guide	10
Design - Wireframes	10
Mentor Mark	
Industry Mentor	10
Lecturer Mentor	10
Individual Mark	
Git Contributions	10
Peer Review	10
Total	**150**

The following penalties will be enforced where the specified conditions are met. The percentage indicated represents the proportion by which the awarded mark will be reduced. For example, a penalty of 50% will halve the final mark (e.g. 80% becomes 40%), while a penalty of 100% will result in a final mark of 0%.

Note: The Git Contributions mark is not determined solely by your Hyperperform score, but by your overall code contribution to the project.

Disclaimer: Mark adjustments may be applied based on discrepancies between individual contributions and overall team performance. This is intended to promote fair and equitable participation across the team, ensuring that the project workload is appropriately shared and does not rest disproportionately on any single member.

PENALTIES	
Condition	**Of total mark**
Git Contributions Mark <40%	50%
Git Contributions Mark <20%	100%
Git - Testing only	50%
Git - No code	100%
Peer review Mark <40%	50%
Peer review Mark <20%	100%
Vibe Coding	100%

We reserve the right to adjust marks where discrepancies are identified. The following are examples of actions that may trigger further review:

- Git commit activity that is heavily concentrated in a short period (e.g. more than 50% of commits occurring within one week prior to the demo).
- Providing inaccurate or misleading information in peer reviews.
- Artificially inflating commit history (e.g. unnecessary or low-value commits).
- Demonstrating dishonesty or misrepresentation during a demo.

7 Working Prototype

Ensure that at least three low-level use cases are implemented and tested for this Demo with an automated testing framework.

8 Project Client

Ensure regular discussions with your project owner to gain approval for the required artefacts. Ensure clients are aware of your GitHub repository.

9 Demo 1 Preparation Checklist

Prepare for Demo 1 by completing the following:

- Demo using a user story to showcase your features.
- Completed a meeting with the client for requirements clarification.
- Sent the demo invite link to your clients/mentors.

- Have a prototype with at least three working use cases.
- Setup a session with your lecturer mentor, industry mentor, and client at least three times per month.

9.1 Extra Tips

- Use PowerPoint, Google Slides, or Canva for presentations.
- Ensure all members are present during the demo in a seamless fashion.
- Time yourself before the demo to ensure you stay within the time limit.